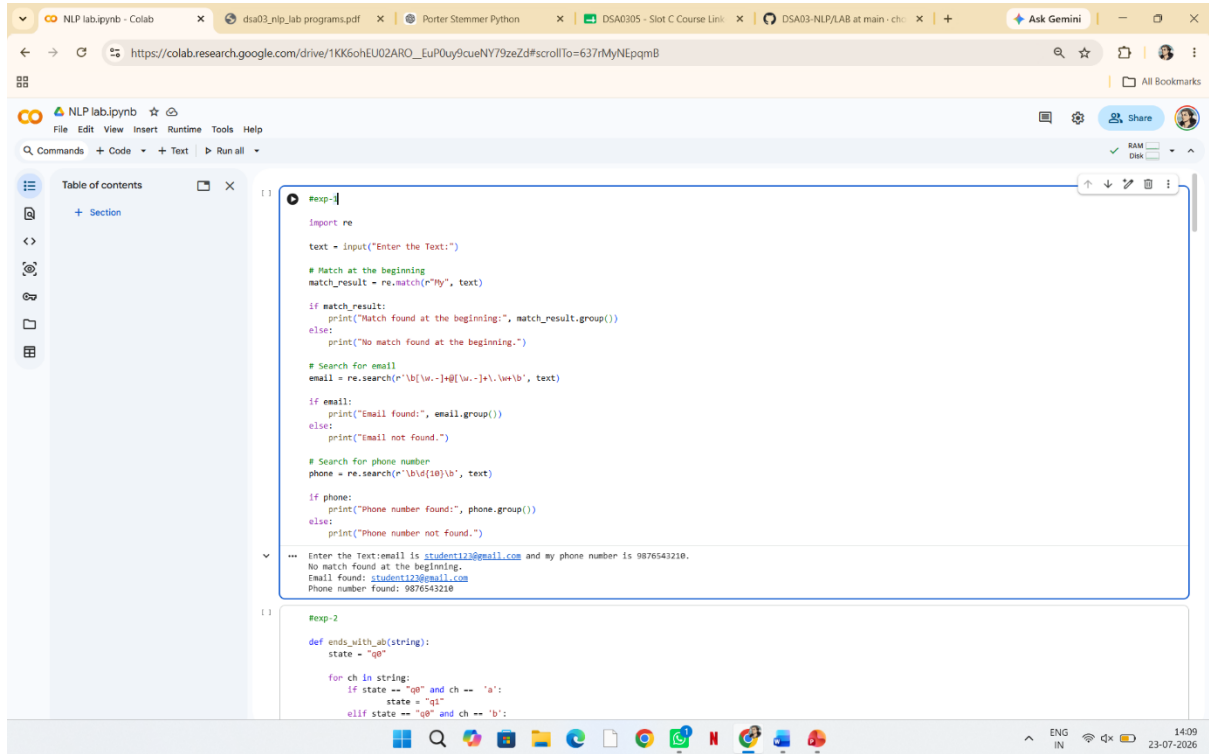


LAB OUTPUTS

EXP-1: -



The screenshot shows a Google Colab notebook titled "NLP lab.ipynb". The code in the first cell, labeled "#exp-1", uses the 're' module to perform pattern matching. It prompts the user to enter text and then checks for a match at the beginning, an email address, and a phone number. The output shows that the input "email is student123@gmail.com and my phone number is 9876543210" results in a match at the beginning, an email found, and a phone number found.

```
#exp-1
import re

text = input("Enter the Text:")

# Match at the beginning
match_result = re.match(r"my", text)

if match_result:
    print("Match found at the beginning:", match_result.group())
else:
    print("No match found at the beginning.")

# Search for email
email = re.search(r'[a-zA-Z0-9]+@[a-zA-Z0-9]+\.[a-zA-Z]+', text)

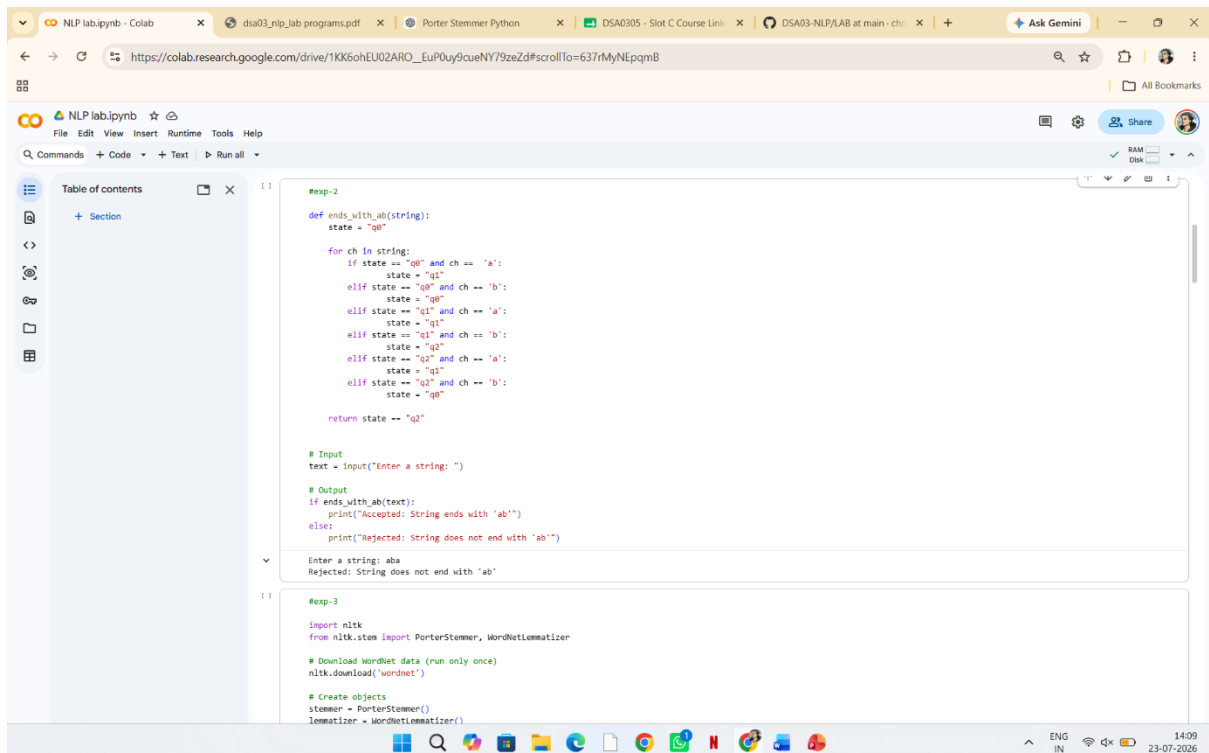
if email:
    print("Email found:", email.group())
else:
    print("Email not found.")

# Search for phone number
phone = re.search(r'\b\d{10}\b', text)

if phone:
    print("Phone number found:", phone.group())
else:
    print("Phone number not found.")
```

Enter the Text:email is student123@gmail.com and my phone number is 9876543210.
No match found at the beginning.
Email found: student123@gmail.com
Phone number found: 9876543210

EXP-2: -



The screenshot shows a Google Colab notebook titled "NLP lab.ipynb". The code in the first cell, labeled "#exp-2", implements a finite state automaton (FSA) to check if a string ends with "ab". The code defines a function 'ends_with_ab' that transitions between states 'q0', 'q1', and 'q2' based on the input characters 'a' and 'b'. The output shows that the input "aba" is rejected because it does not end with "ab". The second cell, labeled "#exp-3", imports NLTK and sets up the PorterStemmer and WordNet lemmatizer.

```
#exp-2
def ends_with_ab(string):
    state = "q0"

    for ch in string:
        if state == "q0" and ch == "a":
            state = "q1"
        elif state == "q0" and ch == "b":
            state = "q0"
        elif state == "q1" and ch == "a":
            state = "q1"
        elif state == "q1" and ch == "b":
            state = "q2"
        elif state == "q2" and ch == "a":
            state = "q1"
        elif state == "q2" and ch == "b":
            state = "q0"

    return state == "q2"

# Input
text = input("Enter a string: ")

# Output
if ends_with_ab(text):
    print("Accepted: String ends with 'ab'")
else:
    print("Rejected: String does not end with 'ab'")
```

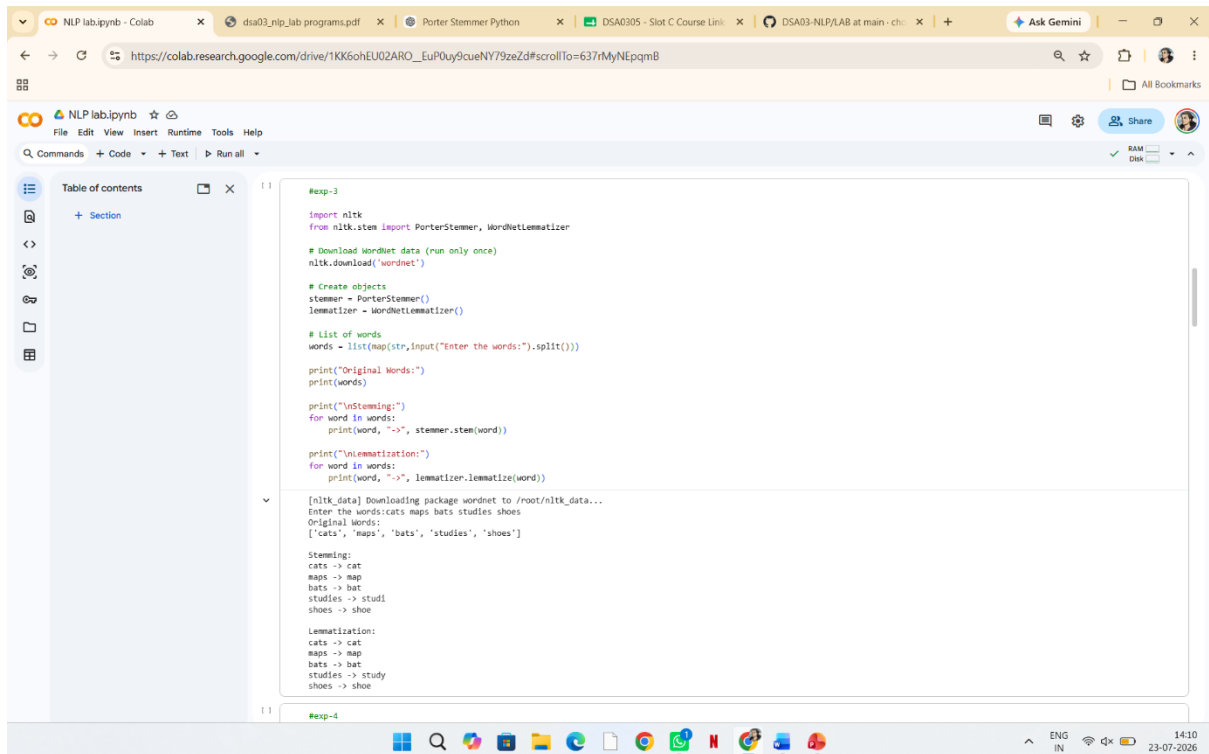
Enter a string: aba
Rejected: String does not end with 'ab'

```
#exp-3
import nltk
from nltk.stem import PorterStemmer, WordNetLemmatizer

# Download WordNet data (run only once)
nltk.download('wordnet')

# Create objects
stemmer = PorterStemmer()
lemmatizer = WordNetLemmatizer()
```

EXP-3: -



The screenshot shows a Google Colab notebook titled "NLP lab.ipynb". The code in the cell is as follows:

```
#exp-3
import nltk
from nltk.stem import PorterStemmer, WordNetLemmatizer

# Download WordNet data (run only once)
nltk.download('wordnet')

# Create objects
stemmer = PorterStemmer()
lemmatizer = WordNetLemmatizer()

# List of words
words = list(map(str, input("Enter the words:").split()))

print("Original Words:")
print(words)

print("\nStemming:")
for word in words:
    print(word, "->", stemmer.stem(word))

print("\nLemmatization:")
for word in words:
    print(word, "->", lemmatizer.lemmatize(word))
```

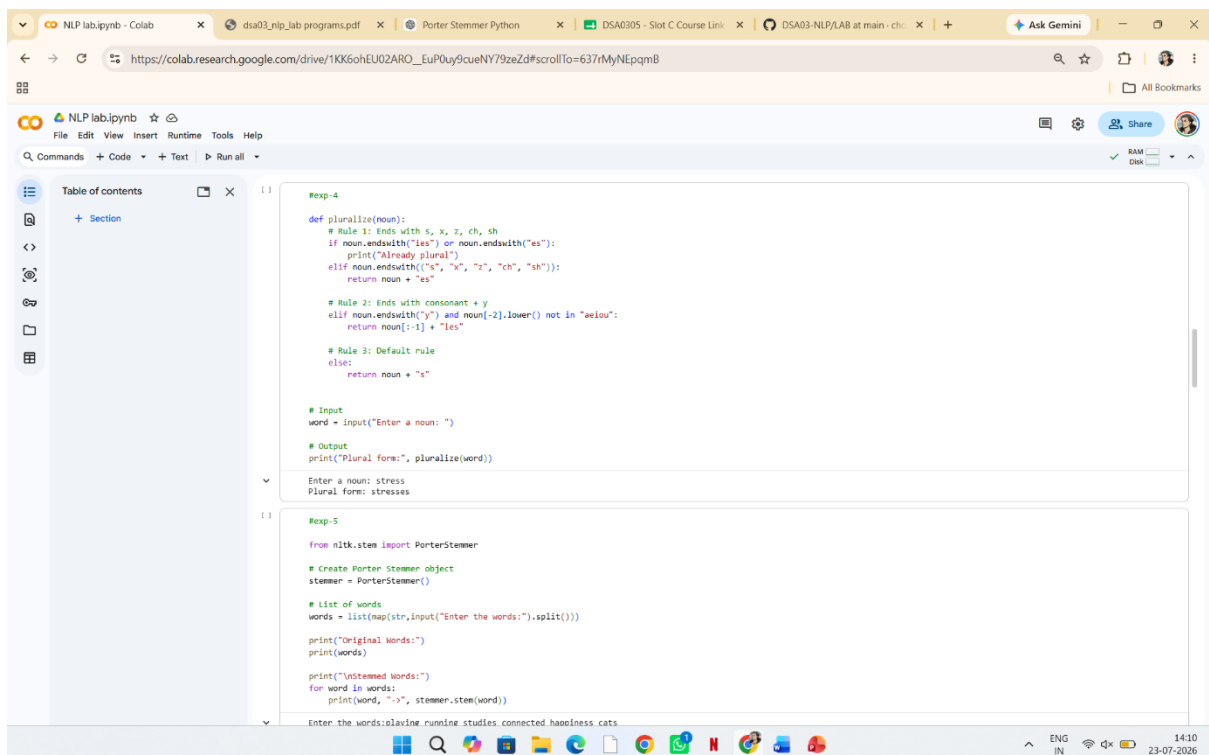
The output of the code is shown below the code cell:

```
[nltk_data] downloading package wordnet to /root/nltk_data...
Enter the words:cats maps bats studies shoes
Original Words:
['cats', 'maps', 'bats', 'studies', 'shoes']

Stemming:
cats -> cat
maps -> map
bats -> bat
studies -> studi
shoes -> shoe

Lemmatization:
cats -> cat
maps -> map
bats -> bat
studies -> study
shoes -> shoe
```

EXP-4: -



The screenshot shows a Google Colab notebook titled "NLP lab.ipynb". The code in the cell is as follows:

```
#exp-4
def pluralize(noun):
    # Rule 1: Ends with s, x, z, ch, sh
    if noun.endswith("ies") or noun.endswith("es"):
        print("Already plural")
    elif noun.endswith(("s", "x", "z", "ch", "sh")):
        return noun + "es"

    # Rule 2: Ends with consonant + y
    elif noun.endswith("y") and noun[-2].lower() not in "aeiou":
        return noun[:-1] + "ies"

    # Rule 3: Default rule
    else:
        return noun + "s"

# Input
word = input("Enter a noun: ")

# Output
print("Plural form:", pluralize(word))
```

The output of the code is shown below the code cell:

```
Enter a noun: stress
Plural form: stresses
```

The code in the next cell is as follows:

```
#exp-5
from nltk.stem import PorterStemmer

# Create Porter Stemmer object
stemmer = PorterStemmer()

# List of words
words = list(map(str, input("Enter the words:").split()))

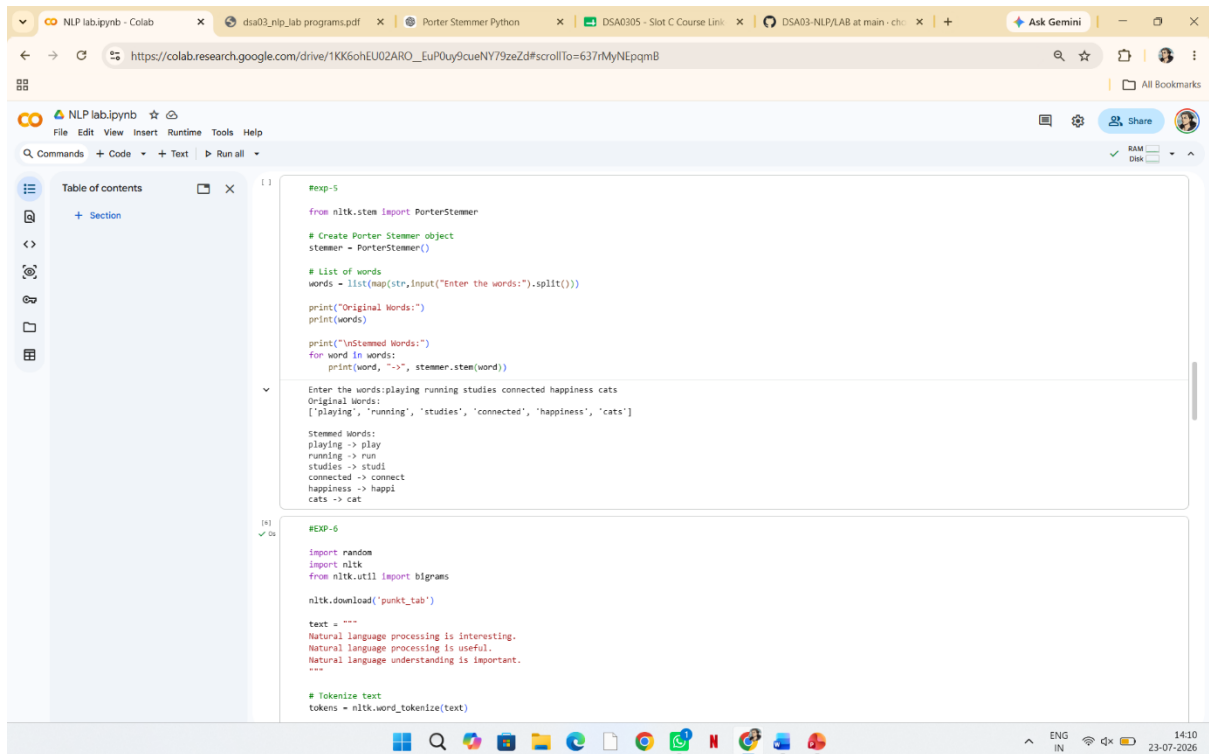
print("Original Words:")
print(words)

print("\nStemmed Words:")
for word in words:
    print(word, "->", stemmer.stem(word))
```

The output of the code is shown below the code cell:

```
Enter the words:online runtime studies connected happiness cats
```

EXP-5: -



```
#exp-5

from nltk.stem import PorterStemmer

# Create Porter Stemmer object
stemmer = PorterStemmer()

# List of words
words = list(map(str, input("Enter the words:").split()))

print("Original Words:")
print(words)

print("\nStemmed Words:")
for word in words:
    print(word, "->", stemmer.stem(word))
```

Enter the words: playing running studies connected happiness cats
Original Words:
['playing', 'running', 'studies', 'connected', 'happiness', 'cats']
Stemmed Words:
playing -> play
running -> run
studies -> studi
connected -> connect
happiness -> happi
cats -> cat

```
#EXP-6

import random
import nltk
from nltk.util import bigrams

nltk.download('punkt_tab')

text = """
Natural language processing is interesting.
Natural language processing is useful.
Natural language understanding is important.
"""

# Tokenize text
tokens = nltk.word_tokenize(text)

# Generate bigrams
bg = list(bigrams(tokens))

# Build bigram dictionary
bigram_dict = {}

for w1, w2 in bg:
    if w1 not in bigram_dict:
        bigram_dict[w1] = []
    bigram_dict[w1].append(w2)

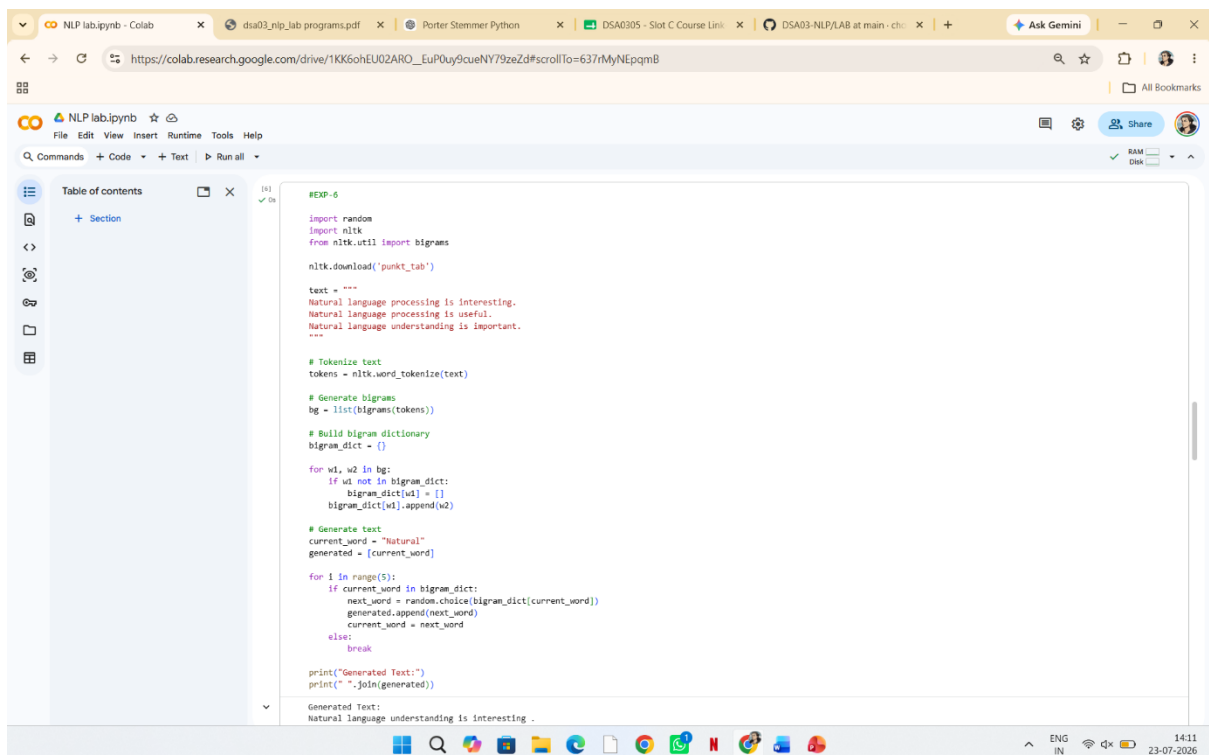
# Generate text
current_word = "Natural"
generated = [current_word]

for i in range(5):
    if current_word in bigram_dict:
        next_word = random.choice(bigram_dict[current_word])
        generated.append(next_word)
        current_word = next_word
    else:
        break

print("Generated Text:")
print(" ".join(generated))
```

Generated Text:
Natural language understanding is interesting .

EXP-6: -



```
#EXP-6

import random
import nltk
from nltk.util import bigrams

nltk.download('punkt_tab')

text = """
Natural language processing is interesting.
Natural language processing is useful.
Natural language understanding is important.
"""

# Tokenize text
tokens = nltk.word_tokenize(text)

# Generate bigrams
bg = list(bigrams(tokens))

# Build bigram dictionary
bigram_dict = {}

for w1, w2 in bg:
    if w1 not in bigram_dict:
        bigram_dict[w1] = []
    bigram_dict[w1].append(w2)

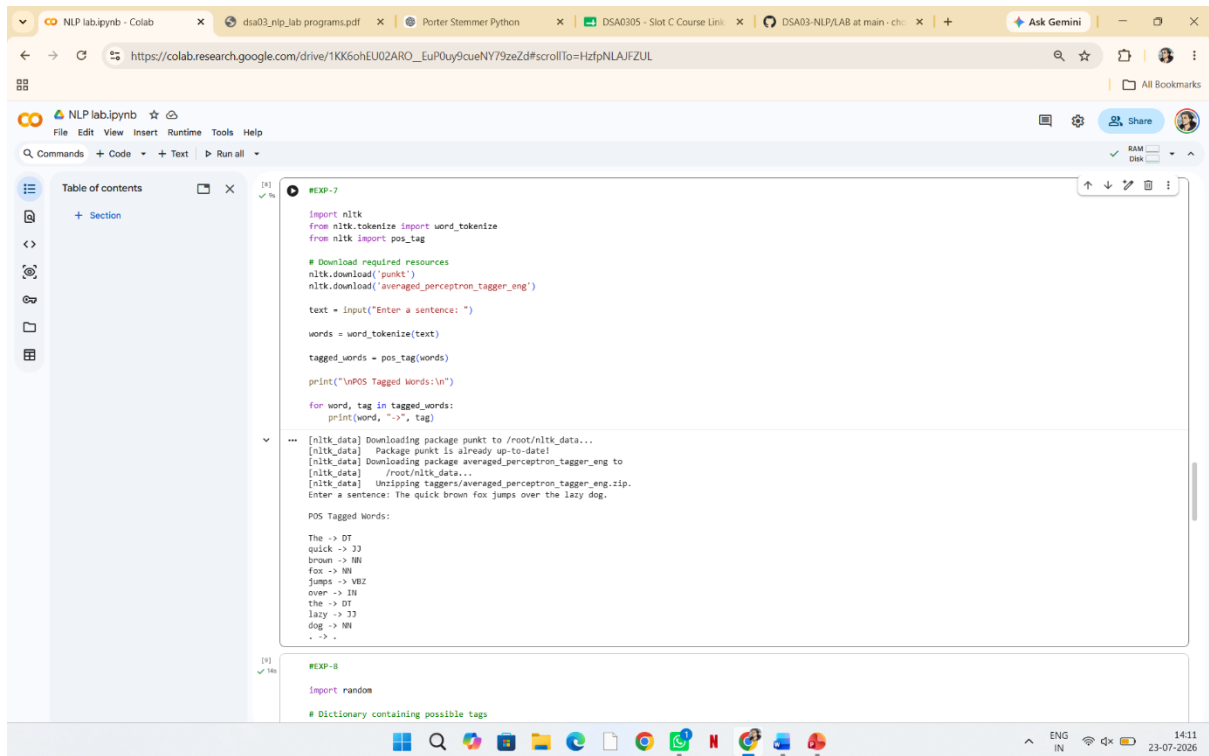
# Generate text
current_word = "Natural"
generated = [current_word]

for i in range(5):
    if current_word in bigram_dict:
        next_word = random.choice(bigram_dict[current_word])
        generated.append(next_word)
        current_word = next_word
    else:
        break

print("Generated Text:")
print(" ".join(generated))
```

Generated Text:
Natural language understanding is interesting .

EXP-7: -



The screenshot shows a Google Colab notebook titled "NLP lab.ipynb". The code in the cell is as follows:

```
import nltk
from nltk.tokenize import word_tokenize
from nltk import pos_tag

# Download required resources
nltk.download('punkt')
nltk.download('averaged_perceptron_tagger_eng')

text = input("Enter a sentence: ")

words = word_tokenize(text)

tagged_words = pos_tag(words)

print("\nPOS Tagged Words:\n")

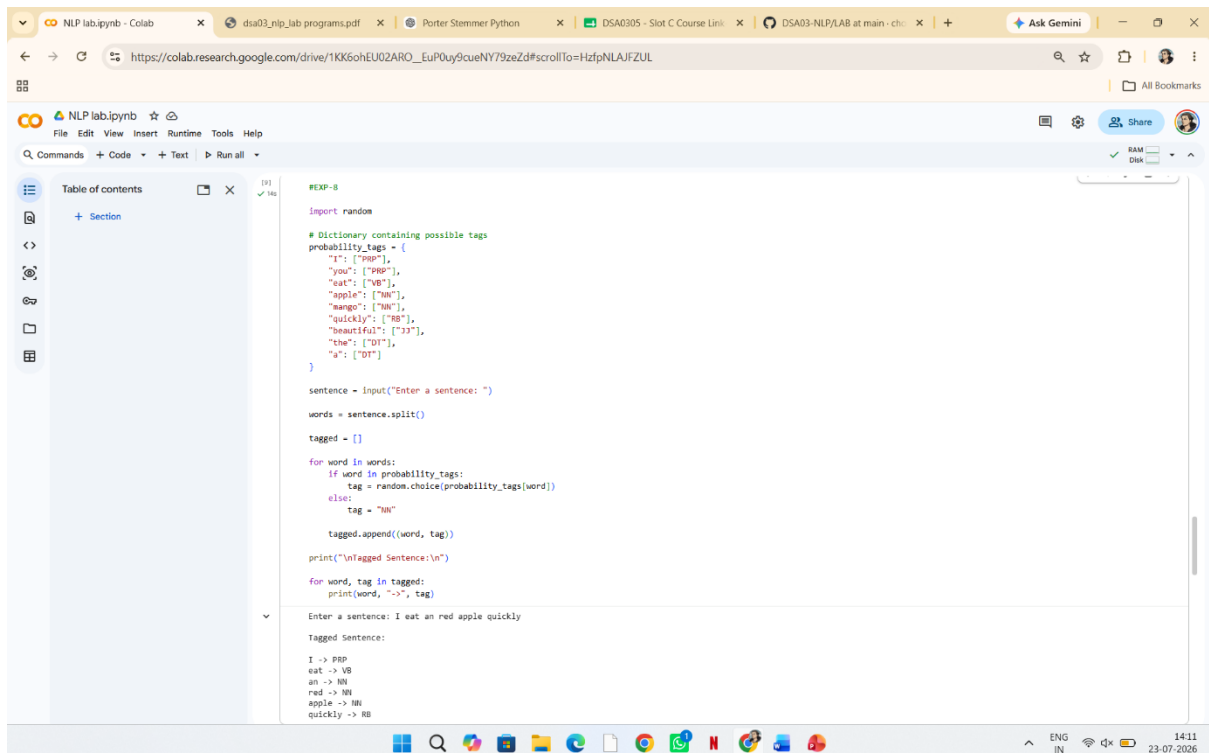
for word, tag in tagged_words:
    print(word, "->", tag)
```

The output of the code is:

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data] Package punkt is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data] /root/nltk_data...
[nltk_data] Unzipping taggers/averaged_perceptron_tagger_eng.zip.
Enter a sentence: The quick brown fox jumps over the lazy dog.

POS Tagged Words:
The -> DT
quick -> JJ
brown -> NN
fox -> NN
jumps -> VBZ
over -> IN
the -> DT
lazy -> JJ
dog -> NN
- -> .
```

EXP-8: -



The screenshot shows a Google Colab notebook titled "NLP lab.ipynb". The code in the cell is as follows:

```
import random

# Dictionary containing possible tags
probability_tags = {
    "I": ["PRP"],
    "you": ["PRP"],
    "eat": ["VB"],
    "apple": ["NN"],
    "mango": ["NN"],
    "quickly": ["RB"],
    "beautiful": ["JJ"],
    "the": ["DT"],
    "a": ["DT"]
}

sentence = input("Enter a sentence: ")

words = sentence.split()

tagged = []

for word in words:
    if word in probability_tags:
        tag = random.choice(probability_tags[word])
    else:
        tag = "NN"

    tagged.append((word, tag))

print("\ntagged Sentence:\n")

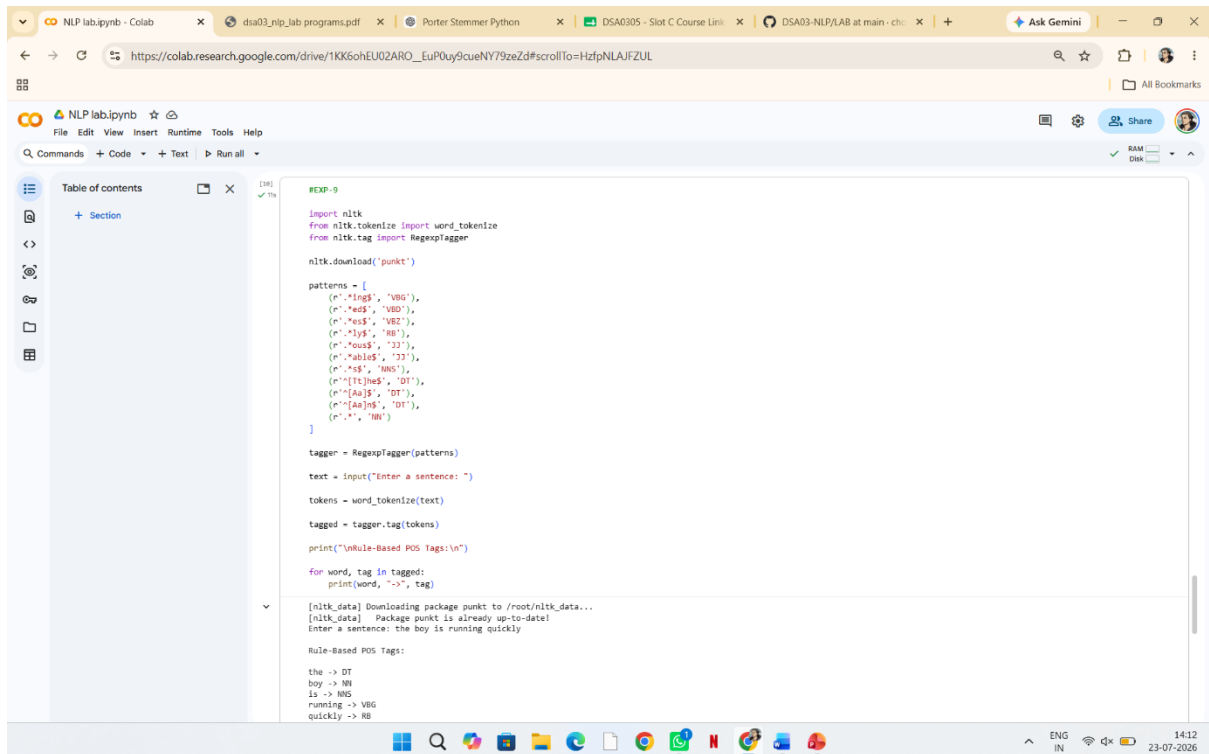
for word, tag in tagged:
    print(word, "->", tag)
```

The output of the code is:

```
Enter a sentence: I eat an red apple quickly

Tagged Sentence:
I -> PRP
eat -> VB
an -> NN
red -> NN
apple -> NN
quickly -> RB
```

EXP-9: -



```
import nltk
from nltk.tokenize import word_tokenize
from nltk.tag import RegexpTagger

nltk.download('punkt')

patterns = [
    (r"^[a-zA-Z]*s$", "NNS"),
    (r"^[a-zA-Z]*ed$", "VBD"),
    (r"^[a-zA-Z]*ing$", "VBG"),
    (r"^[a-zA-Z]*ly$", "RB"),
    (r"^[a-zA-Z]*ous$", "JJ"),
    (r"^[a-zA-Z]*able$", "JJ"),
    (r"^[a-zA-Z]*ness$", "NN"),
    (r"^[a-zA-Z]*the$", "DT"),
    (r"^[a-zA-Z]*a$", "DT"),
    (r"^[a-zA-Z]*an$", "DT"),
    (r"^[a-zA-Z]*$", "NN")
]

tagger = RegexpTagger(patterns)

text = input("Enter a sentence: ")

tokens = word_tokenize(text)

tagged = tagger.tag(tokens)

print("\nRule-Based POS Tags:\n")

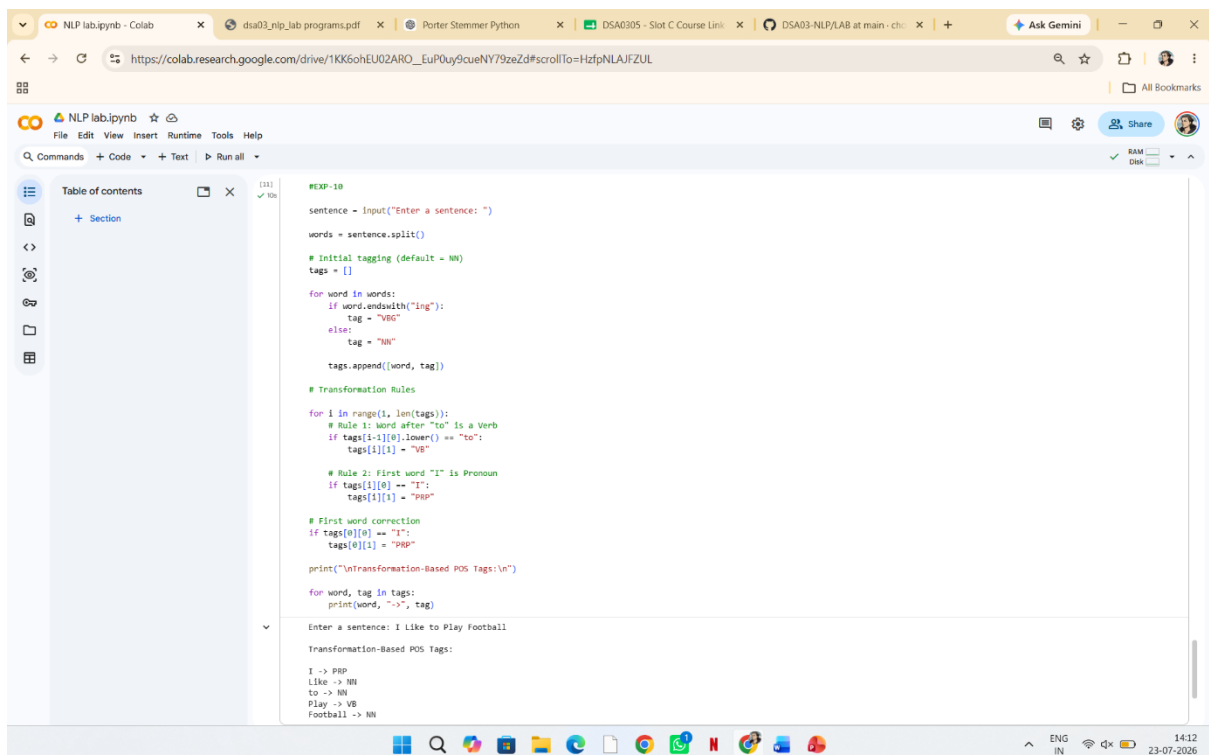
for word, tag in tagged:
    print(word, "->", tag)
```

[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data] Package punkt is already up-to-date!
Enter a sentence: the boy is running quickly

Rule-Based POS Tags:

the -> DT
boy -> NN
is -> VB
running -> VBG
quickly -> RB

EXP-10: -



```
SENTENCE = input("Enter a sentence: ")

words = sentence.split()

# Initial tagging (default = NN)
tags = []

for word in words:
    if word.endswith("ing"):
        tag = "VBG"
    else:
        tag = "NN"
    tags.append([word, tag])

# Transformation Rules

for i in range(1, len(tags)):
    # Rule 1: Word after "to" is a Verb
    if tags[i-1][0].lower() == "to":
        tags[i][1] = "VB"

    # Rule 2: First word "I" is Pronoun
    if tags[i][0] == "I":
        tags[i][1] = "PRP"

# First word correction
if tags[0][0] == "I":
    tags[0][1] = "PRP"

print("\nTransformation-Based POS Tags:\n")

for word, tag in tags:
    print(word, "->", tag)
```

Enter a sentence: I like to play football

Transformation-Based POS Tags:

I -> PRP
like -> VB
to -> TO
play -> VB
football -> NN